



Procesador RISC-V con Arquitectura Pipelined (Segmentada)

Informe Técnico de Diseño e Implementación

Curso: Arquitectura de Computadoras 2026-1
Sección: 4
Universidad: Universidad de Ingeniería y Tecnología (UTEC)
Repositorio: github.com/Auky216/Microarchitecture-Pipelined

Integrantes:

Isaac Emanuel Javier Simeón Sarmiento

isaac.simeon@utec.edu.pe

Alexandro Martin Chamocho Gumbi Gutierrez

alexandro.chamocho@utec.edu.pe

Adrian Antonio Auqui Perez

adrian.auqui@utec.edu.pe

25 de junio de 2026

Índice

1. Teoría del Procesador Pipeline: Datapath y Control (0.5 pts)	3
1.1. ¿Qué es un Procesador Pipelined?	3
1.2. Las 5 Etapas del Pipeline	3
1.3. Diagrama Temporal del Pipeline	3
1.4. ISA Implementada: RISC-V RV32I	4
1.5. El Camino de Control en el Pipeline	4
2. Implementación de Instrucciones (Cuadro 1) (1.0 pts)	5
2.1. Descripción de Cambios en el Código	7
2.2. Diagrama de Datapath Final (Básico)	7
2.3. Los Registros Pipeline	8
2.3.1. ¿Qué son y por qué son necesarios?	8
2.3.2. Tres Tipos de Flip-Flops	9
2.3.3. ¿Por qué IF/ID usa flopenrc y MEM/WB usa flopr?	9
2.4. Módulo Top-Level: <code>riscv_pipeline</code>	9
2.5. Etapa 1: Fetch (IF)	10
2.5.1. Componentes Activos	10
2.5.2. Pseudocódigo	10
2.6. Etapa 2: Decode (ID)	11
2.7. Componentes Activos	11
2.7.1. Unidad de Control	11
2.7.2. Pseudocódigo de Decode	12
3. Etapa 3: Execute (EX)	12
3.1. Componentes Activos	13
3.2. Pseudocódigo Completo del Execute	13
4. Etapas Memory (MEM) y Writeback (WB)	14
4.1. Componentes Activos	15
4.2. Pseudocódigo de ambas etapas	15
5. Programa ISA sin Dependencias (1.0 pts)	16
5.1. Código Ensamblador	16
5.2. Waveform del Programa 1	16
5.3. Mostrar con Resultados Intermedios que el Programa Funciona Correctamente	17
6. Teoría de Hazard Unit y Evidencia de Fallos (0.5 pts)	17
6.1. Teoría de Control de Riesgos en Procesadores Segmentados	17
6.2. Evidencia: Sin Hazard Unit los Programas Fallan	17
6.2.1. Programa 2 SIN Hazard Unit (Forwarding)	18
6.2.2. Programa 3 SIN Hazard Unit (Stalling)	18
6.2.3. Programa 4 SIN Hazard Unit (Flushing)	19
7. Implementación del Hazard Unit (1.0 pts)	19
7.1. Descripción de Cambios en el Código	19

7.2. Mecanismo 1: Forwarding (Reenvío de Datos)	20
7.3. Mecanismo 2: Stall (Load-Use Hazard)	20
7.4. Mecanismo 3: Flush (Control Hazard por Saltos)	21
7.5. Especificación de Señales de Control de Riesgos	21
7.6. Diagrama de Datapath Final (con Hazard Unit)	21
8. Programas de Prueba (CON Hazard Unit) (1.0 pts)	23
8.1. Programa 2: Forwarding	23
8.2. Programa 3: Stalling	23
8.3. Programa 4: Flushing	24
8.4. Programa Extra: Set Completo de Operaciones y Validación de la ISA y Hazard Unit	24
9. Jerarquía de Archivos	27
10. Conclusiones	27

1 Teoría del Procesador Pipeline: Datapath y Control (0.5 pts)

1.1 ¿Qué es un Procesador Pipelined?

Un procesador **pipelined** (segmentado) divide la ejecución de cada instrucción en varias etapas independientes que trabajan en paralelo. Mientras una instrucción está siendo decodificada, la siguiente ya está siendo leída de la memoria, y la anterior ya está realizando su operación aritmética.

Teoría del Paralelismo a Nivel de Instrucción (ILP): El pipelining es la forma más pura de ILP. En un procesador **de ciclo único**, cada instrucción tarda un ciclo completo (cuyo reloj se calibra según la instrucción más lenta, como `lw`). La latencia total y el periodo del reloj limitan drásticamente la frecuencia (MHz/GHz). En contraste, el pipeline divide ese largo camino combinacional en **5 etapas** más cortas separadas por registros.

Esto reduce drásticamente el periodo del reloj y aumenta la frecuencia. Idealmente, el **throughput** (rendimiento) se incrementa por un factor igual al número de etapas ($k = 5$). Aunque la *latencia* de una instrucción individual no mejora (incluso empeora ligeramente por el retardo de los registros), el hecho de que termine **una instrucción por cada ciclo de reloj** hace que el procesador sea inmensamente más rápido a gran escala.

1.2 Las 5 Etapas del Pipeline

1. **Fetch (IF):** Lectura de la instrucción desde la memoria de instrucciones.
2. **Decode (ID):** Decodificación de la instrucción, lectura de registros y generación de señales de control.
3. **Execute (EX):** Ejecución de la operación aritmético-lógica en la ALU.
4. **Memory (MEM):** Acceso a la memoria de datos.
5. **Writeback (WB):** Escritura del resultado final en el banco de registros.

1.3 Diagrama Temporal del Pipeline

El siguiente diagrama muestra cómo 5 instrucciones se solapan en el pipeline:

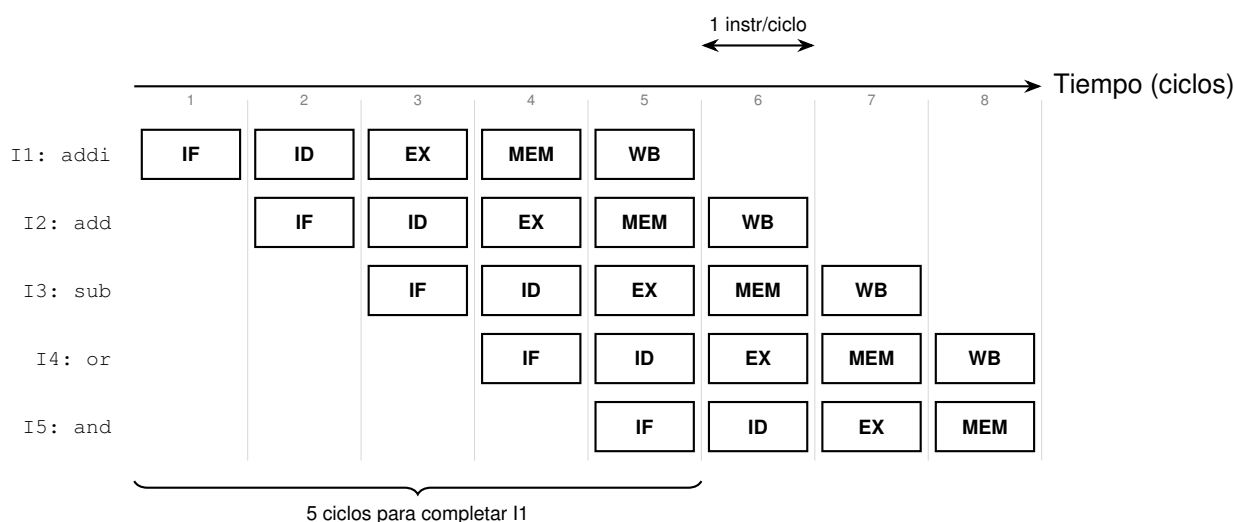


Figura 1: Diagrama temporal: cada instrucción tarda 5 ciclos, pero se completa 1 instrucción por ciclo.

1.4 ISA Implementada: RISC-V RV32I

Tipo	Instrucciones	Opcode	Ejemplo
R-type	add, sub, and, or, sll, slt	0110011	add x4, x2, x3
I-type	addi, lw, ori, slti	0010011 / 0000011	addi x2, x0, 5
S-type	sw	0100011	sw x7, 100(x3)
B-type	beq	1100011	beq x4, x0, L
J-type	jal	1101111	jal x3, func

Tabla 1: Instrucciones soportadas por el procesador.

1.5 El Camino de Control en el Pipeline

A diferencia de un procesador monociclo donde las señales de control generadas por la Unidad de Control se aplican de forma inmediata a todos los componentes, en un procesador segmentado (pipelined) el camino de control debe ser también segmentado.

La decodificación de la instrucción y la generación de señales se realizan exclusivamente en la etapa de decodificación (**Decode, ID**). No obstante, las acciones que estas señales comandan ocurren en etapas posteriores. Por lo tanto, las señales de control deben ser almacenadas en los registros de pipeline correspondientes y propagarse en sincronía con la instrucción asociada a lo largo del procesador:

- **Señales para Execute (EX):** Controlan operaciones en la ALU (ALUControl, ALUSrc). Pasan por el registro IF/ID y se almacenan en el registro ID/EX para aplicarse en Execute. Luego se descartan al pasar a la siguiente etapa.

- **Señales para Memory (MEM):** Controlan lecturas o escrituras en la memoria de datos (`MemWrite`). Deben viajar por el registro `ID/EX` hasta la etapa `EX`, y luego registrarse en el buffer `EX/MEM` para activarse durante la etapa `MEM`.
- **Señales para Writeback (WB):** Indican la escritura en el banco de registros (`RegWrite`, `ResultSrc`). Deben viajar consecutivamente por todos los registros del pipeline (`ID/EX`, `EX/MEM` y `MEM/WB`) hasta llegar a la etapa de Writeback para completar la operación.

Este esquema de control registrado asegura que las señales de habilitación de escritura o selección de multiplexores correspondan estrictamente a la instrucción que se encuentra activa en cada sección del datapath en un instante de tiempo dado.

2 Implementación de Instrucciones (Cuadro 1) (1.0 pts)

A continuación se presenta el listado de las instrucciones que se solicitó evaluar e implementar en el pipeline base (Cuadro 1) junto con un análisis detallado de su soporte en la arquitectura actual del procesador:

Instrucción	Descripción	Soportada	Detalle / Estado en el Proyecto
lw rd, imm(rs1)	Load word	Sí	Opcode 0000011 decodificado en el controlador.
addi rd, rs1, imm	Add immediate	Sí	Decodificado. ALU ejecuta suma (ALUControl = 0000).
slli rd, rs1, shamt	Shift left logical imm.	Sí	Decodificado. ALU ejecuta desplazamiento izquierdo (0001).
xori rd, rs1, imm	XOR immediate	Sí	Decodificado. ALU ejecuta XOR (0100).
srli rd, rs1, shamt	Shift right logical imm.	Sí	Decodificado. ALU ejecuta desplazamiento derecho lógico (0101).
srai rd, rs1, shamt	Shift right arithmetic imm.	Sí	Decodificado. ALU ejecuta desp. derecho aritmético (1101).
ori rd, rs1, imm	OR immediate	Sí	Decodificado. ALU ejecuta OR (0110).
andi rd, rs1, imm	AND immediate	Sí	Decodificado. ALU ejecuta AND (0111).
sw rs2, imm(rs1)	Store word	Sí	Opcode 0100011 decodificado en el controlador.
add rd, rs1, rs2	Add	Sí	R-type decodificado. ALU ejecuta suma.
sub rd, rs1, rs2	Subtract	Sí	R-type decodificado. ALU ejecuta resta (ALUControl = 1000).
sll rd, rs1, rs2	Shift left logical	Sí	R-type decodificado. ALU ejecuta desplazamiento izquierdo.
xor rd, rs1, rs2	XOR	Sí	R-type decodificado. ALU ejecuta XOR.
srl rd, rs1, rs2	Shift right logical	Sí	R-type decodificado. ALU ejecuta desp. derecho lógico.
sra rd, rs1, rs2	Shift right arithmetic	Sí	R-type decodificado. ALU ejecuta desp. derecho aritmético.
or rd, rs1, rs2	OR	Sí	R-type decodificado. ALU ejecuta OR.
and rd, rs1, rs2	AND	Sí	R-type decodificado. ALU ejecuta AND.
lui rd, imm	Load upper immediate	Sí	Decodificado (ImmSrc=3'b100). ALU en modo pasarela del operando B (ALUControl=1111).
beq rs1, rs2, offset	Branch if equal	Sí	Decodificado. Comparación por resta y decisión por ZeroE.
bne rs1, rs2, offset	Branch if not equal	Sí	Decodificado. Lógica combinacional en Execute evalúa desigualdad (ZeroE).
blt rs1, rs2, offset	Branch if less than	Sí	Decodificado. Lógica en Execute evalúa comparación signada menor que ($\$signed(SrcAE) < \$signed(SrcBE)$).
bge rs1, rs2, offset	Branch if greater or equal	Sí	Decodificado. Lógica en Execute evalúa comparación signada mayor o igual ($\$signed(SrcAE) \geq \$signed(SrcBE)$).
jalr rd, rs1, imm	Jump and link register	Sí	Decodificado. Dirección de salto calculada con base en registro (SrcAE), con el bit menos significativo en 0.
jal rd, offset	Jump and link	Sí	Opcode 1101111 decodificado. Escribe PC+4 en banco de registros.

Tabla 2: Instrucciones evaluadas e implementadas en el pipeline y su estado de soporte.

2.1 Descripción de Cambios en el Código

Para transformar la arquitectura de ciclo único a una segmentada (pipelined) que soporte el conjunto de instrucciones del Cuadro 1 sin degradar la correctitud, se realizaron los siguientes cambios estructurales en el código Verilog:

1. **Segmentación de Módulos en el Datapath:** La lógica combinacional monolítica se dividió en 5 etapas independientes ubicadas en la carpeta `Datapath/`:
 - `fetch.v`: Controla el Program Counter (PC) y accede a la memoria de instrucciones `imem.v`.
 - `decode.v`: Decodifica operandos, lee del banco de registros (`regfile.v`) y extiende inmediatos.
 - `execute.v`: Procesa operaciones mediante la ALU y calcula la condición y dirección del branch.
 - `memory.v`: Maneja la lectura asíncrona y escritura síncrona en la memoria de datos `dmem.v`.
 - `writeback.v`: Selecciona el dato final a guardar en el Register File.
2. **Control Pipelined (Control Pipeline):** Las señales generadas por la unidad de control en Decode (`controller.v`) se segmentaron agregando registros pipeline. Las señales viajan junto con la instrucción por el datapath (`RegWriteD` → `RegWriteE` → `RegWriteM` → `RegWriteW`).
3. **Propagación de Señales de Instrucción (Debug):** Se implementó el tránsito de las instrucciones completas de 32 bits por cada etapa (`InstrD`, `InstrE`, `InstrM`, `InstrW`). Esto permite visualizar en el testbench y en GTKWave exactamente qué instrucción se encuentra en ejecución o acceso a memoria en cada ciclo.
4. **Dimensionamiento del Registro del Pipeline:** Al inyectar la señal de instrucción de 32 bits en las etapas, se reajustaron los anchos de los flip-flops del pipeline (`floprr/flopenrc`):
 - **Registro ID/EX:** De 186 bits a 218 bits (agregando `InstrD`).
 - **Registro EX/MEM:** De 105 bits a 137 bits (agregando `InstrE`).
 - **Registro MEM/WB:** De 104 bits a 136 bits (agregando `InstrM`).
5. **Control del ALU a 4 bits:** En la unidad de control (`controller.v`), se configuró un bus de 4 bits para la ALU. Esto permite discriminar correctamente la instrucción `add` de `sub` (mediante el bit de `funct7`) y asegurar que el sumador/restador interno funcione sin conflictos en saltos y cargas.

2.2 Diagrama de Datapath Final (Básico)

A continuación, se presenta el esquema del datapath segmentado básico para la ejecución de instrucciones secuenciales (sin incluir la Hazard Unit):

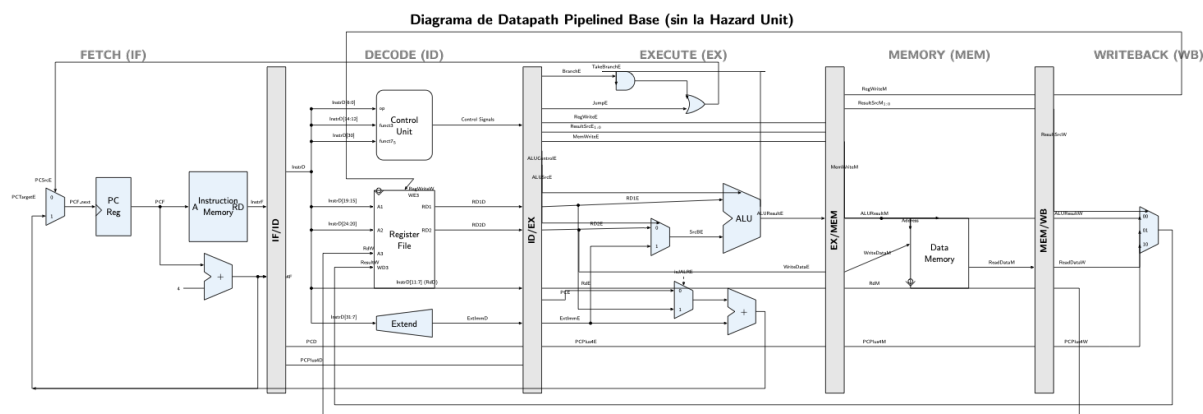


Figura 2: Diagrama del Datapath Pipelined básico del procesador (sin la Hazard Unit).

El datapath básico funciona dividiendo el camino crítico combinacional de la arquitectura de ciclo único en cinco etapas más cortas y balanceadas de forma paralela. En este diseño sin la Hazard Unit, el procesador opera correctamente bajo la suposición de que no existen dependencias de datos ni de control entre instrucciones consecutivas (o que estas se resuelven agregando instrucciones `nop` manuales en el ensamblador). Cada una de las etapas (IF, ID, EX, MEM, WB) ejecuta de forma concurrente una instrucción gracias a los registros de acoplamiento (IF/ID, ID/EX, EX/MEM, MEM/WB), los cuales retienen los operandos y las señales de control correspondientes. Esto permite completar la ejecución de una instrucción en cada ciclo de reloj (IPC ideal = 1) una vez que se ha llenado el pipeline.

2.3 Los Registros Pipeline

2.3.1 ¿Qué son y por qué son necesarios?

Los registros pipeline **separan físicamente** una etapa de la siguiente. Sin ellos, las señales se mezclarían. Se ubican entre cada par de etapas.

2.3.2 Tres Tipos de Flip-Flops

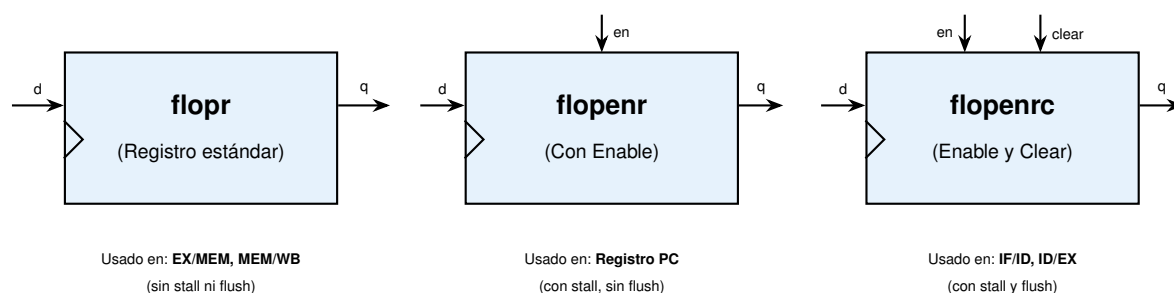


Figura 3: Los tres tipos de flip-flops usados. Observa el triángulo símbolo de la señal de reloj sincronizada por flanco.

2.3.3 ¿Por qué IF/ID usa flopenrc y MEM/WB usa flopr?

Las etapas tempranas (Fetch, Decode, Execute) son **especulativas**: las instrucciones pueden ser descartadas (flush) si un salto cambia el flujo, o detenidas (stall) si hay una dependencia de datos sin resolver.

Las etapas tardías (Memory, Writeback) ejecutan instrucciones que ya **pasaron todos los filtros**: ya se resolvieron los saltos y los riesgos. Nunca hay razón para detenerlas ni vaciarlas, de ahí que solo requieran un **flopr** básico.

```

1 // flopenrc: Tiene clk, reset, enable y clear
2 always @(posedge clk):
3     if (reset)      q = 0;
4     else if (clear) q = 0; // FLUSH: inserta burbuja NOP
5     else if (en)    q = d; // Avanza normalmente
6     // else if (!en): q no cambia (CONGELADO - STALL)

```

Listing 1: Comportamiento de los registros pipeline

2.4 Módulo Top-Level: riscv_pipeline

El módulo `riscv_pipeline` es el **ensamblaje principal** que conecta las 5 etapas con la Hazard Unit. Es puramente estructural, cableando todos los submódulos.

```

1 module riscv_pipeline(clk, reset):
2     // 1. Instanciar Fetch
3     fetchStage(clk, reset, StallF, PCSrcE, PCTargetE ...);
4
5     // 2. Instanciar Decode
6     decodeStage(clk, reset, StallD, FlushD, ...);
7
8     // 3. Instanciar Execute
9     executeStage(clk, reset, FlushE, ForwardAE, ForwardBE, ...);
10
11    // 4. Instanciar Memory
12    memStage(clk, reset, execute_signals ...);
13
14    // 5. Instanciar Writeback

```

```

15  wbStage(clk, reset, memory_signals ...);
16
17  // 6. Instanciar Hazard Unit
18  hazardUnit(Rs1D, Rs2D, Rs1E, Rs2E, RdE, RdM, RdW, ...);

```

Listing 2: Pseudocódigo del módulo top-level

2.5 Etapa 1: Fetch (IF)

2.5.1 Componentes Activos

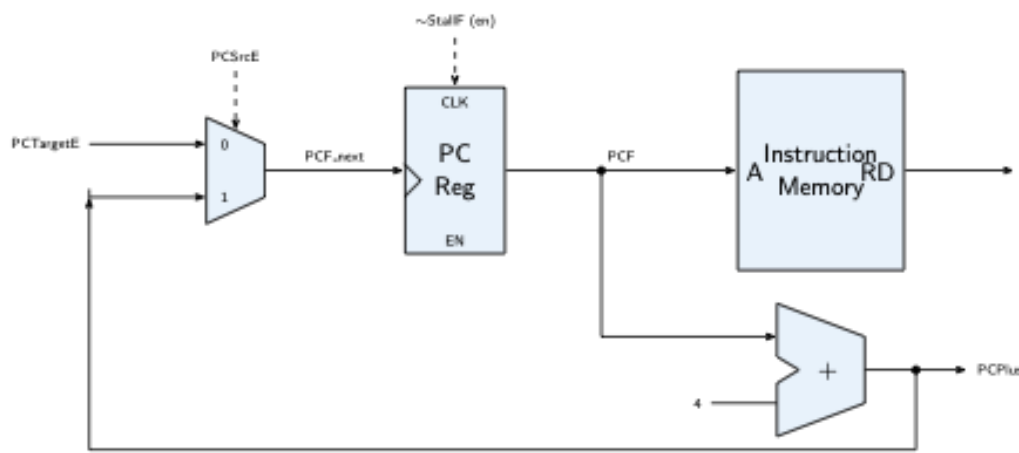


Figura 4: Etapa Fetch aislada con sus componentes operativos.

2.5.2 Pseudocódigo

```

1  FETCH(clk, reset, StallF, PCSrcE, PCTargetE):
2    // 1. Seleccionar el proximo PC
3    if (PCSrcE == 1):
4      PCF_next = PCTargetE;      // Salto tomado
5    else:
6      PCF_next = PCPlus4F;      // Secuencial (PC + 4)
7
8    // 2. Actualizar el PC (solo si no hay stall)
9    always @(posedge clk):
10     if (reset)      PCF = 0;
11     else if (!StallF) PCF = PCF_next;
12
13    // 3. Leer la instruccion de la memoria
14    InstrF = instruction_memory[PCF / 4];
15
16    // 4. Calcular PC + 4
17    PCPlus4F = PCF + 4;

```

Listing 3: Pseudocódigo de la etapa Fetch

2.6 Etapa 2: Decode (ID)

2.7 Componentes Activos

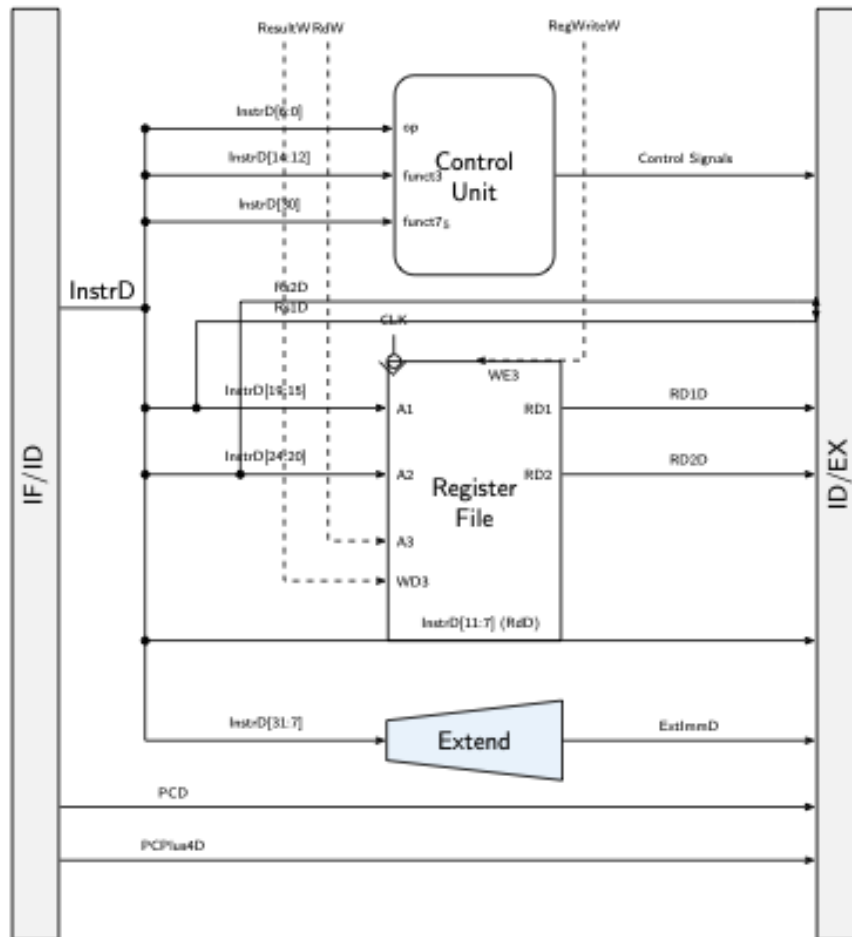


Figura 5: Etapa Decode aislada. Incluye el banco de registros y unidad de control.

2.7.1 Unidad de Control

La Unidad de Control recibe el opcode, `funct3` y `funct7` de la instrucción y genera todas las señales de control.

Tipo	Opcode	RegW	ImmSrc	ALUSrc	MemW	ResSrc	Branch	ALUOp
lw	0000011	1	000	1	0	01	0	00
sw	0100011	0	001	1	1	–	0	00
R-type	0110011	1	xxx	0	0	00	0	10
I-type	0010011	1	000	1	0	00	0	10
beq	1100011	0	010	0	0	–	1	01
jal	1101111	1	011	0	0	10	0	xx
lui	0110111	1	100	1	0	00	0	11
jalr	1100111	1	000	1	0	10	0	00

Tabla 3: Tabla de verdad de la unidad de control principal.

Nota crítica sobre el ALUControl: El `ALUControl` usa **4 bits**, no 3. El bit [3] diferencia ADD (0000) de SUB (1000). Si se truncara a 3 bits, la resta se interpretaría como suma y fallaría la operación `sub` o la lógica de comparación de `beq`.

2.7.2 Pseudocódigo de Decode

```

1 DECODE(clk, reset, StallD, FlushD, InstrF, PCF, PCPlus4F):
2   // 1. Registros IF/ID (flopenrc: con stall y flush)
3   always @(posedge clk):
4     if (reset || FlushD):
5       InstrD = 0; PCD = 0; PCPlus4D = 0;    // Insertar NOP
6     else if (!StallD):
7       InstrD = InstrF; PCD = PCF; PCPlus4D = PCPlus4F;
8
9   // 2. Extraer campos
10  Rs1D = InstrD[19:15]; Rs2D = InstrD[24:20]; RdD = InstrD[11:7];
11
12  // 3. Leer banco de registros (combinacional)
13  RD1D = (Rs1D != 0) ? registers[Rs1D] : 0;
14  RD2D = (Rs2D != 0) ? registers[Rs2D] : 0;
15
16  // 4. Generar senales de control
17  {RegWroteD, MemWroteD, ALUSrcD, ...} = decode_control(InstrD);
18
19  // 5. Extender inmediato
20  ExtImmD = extend_sign(InstrD[31:7], ImmSrcD);

```

Listing 4: Pseudocódigo de la etapa Decode

3 Etapa 3: Execute (EX)

La etapa Execute es el núcleo de cálculo del procesador. Es aquí donde la ALU (Arithmetic Logic Unit) procesa los datos y toma decisiones críticas para el flujo del programa.

Teoría Operativa: A diferencia de un procesador de ciclo único donde la decisión de saltar (Branch) se evalúa en Decode, en este pipeline se evalúa en Execute. La ALU realiza una resta (`sub`) entre los dos operandos y levanta la bandera `ZeroE` si son iguales.

Esta bandera, al operarse mediante una compuerta AND con la señal `BranchE`, determina la señal `PCSrcE` (Branch Taken). Este retraso arquitectónico significa que el salto no se resuelve hasta el tercer ciclo, lo que introduce una penalización de 2 ciclos (burbujas) en caso de que el salto sea tomado, los cuales son gestionados puramente por la *Hazard Unit*.

3.1 Componentes Activos

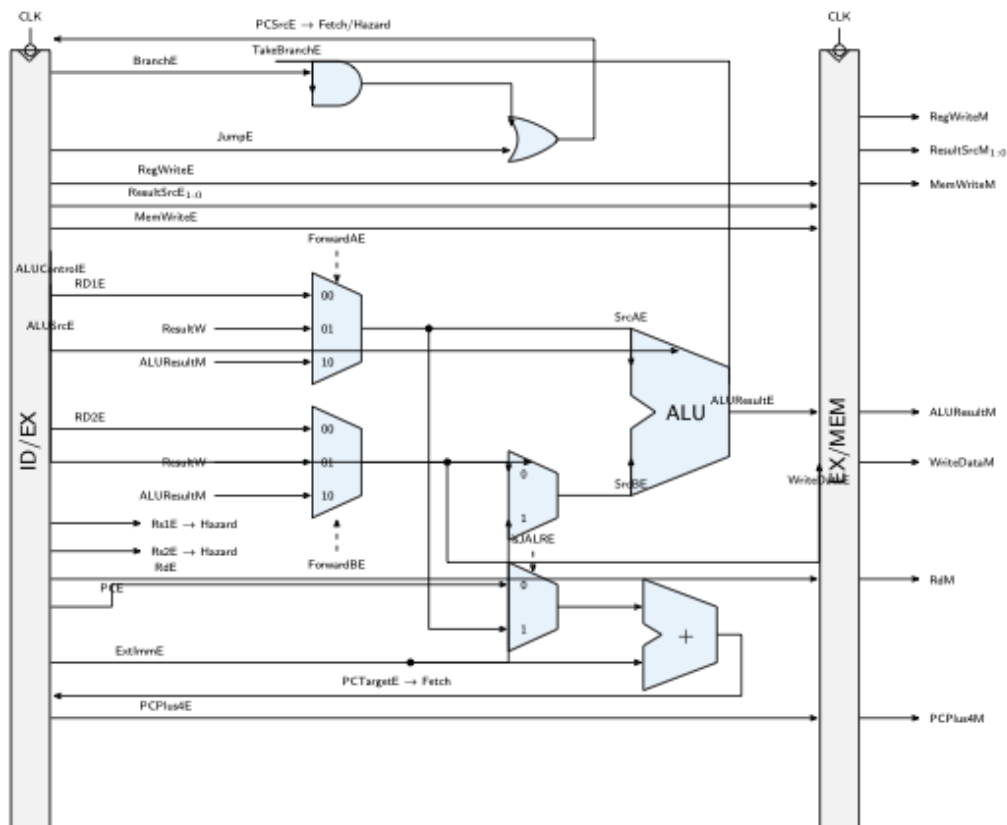


Figura 6: Etapa Execute. Muestra los multiplexores de Forwarding integrados antes de la ALU.

3.2 Pseudocódigo Completo del Execute

```

1 EXECUTE(clk, reset, FlushE, ForwardAE, ForwardBE,
2         control_signals_D, data_D, ALUResultM, ResultW):
3
4 // 1. Registro ID/EX (flopenrc: sin stall, con flush)
5 always @(posedge clk):
6     if (reset || FlushE):
7         all_signals = 0; // Insertar burbuja NOP
8     else:
9         all_signals = decode_signals;
10

```

```

11 // 2. Forwarding Mux A
12 if (ForwardAE == 2'b10):      SrcAE = ALUResultM; // de Memory
13 else if (ForwardAE == 2'b01): SrcAE = ResultW;    // de Writeback
14 else:                        SrcAE = RD1E;        // del registro
15
16 // 3. Forwarding Mux B
17 if (ForwardBE == 2'b10):      WriteDataE = ALUResultM;
18 else if (ForwardBE == 2'b01): WriteDataE = ResultW;
19 else:                        WriteDataE = RD2E;
20
21 // 4. Selección del operando B de la ALU
22 if (ALUSrcE == 1): SrcBE = ExtImmE; // Inmediato
23 else:              SrcBE = WriteDataE; // Registro
24
25 // 5. Ejecutar ALU
26 ALUResultE = ALU(SrcAE, SrcBE, ALUControlE);
27 ZeroE = (ALUResultE == 0);
28
29 // 6. Calcular dirección de salto (Multiplexación JALR / Branch)
30 isJALRE = (InstrE[6:0] == 7'b1100111);
31 PCTargetBaseE = isJALRE ? SrcAE : PCE;
32 PCTargetE = (PCTargetBaseE + ExtImmE) & ~32'd1; // Limpiar LSB
33
34 // 7. Evaluar condición de salto según funct3 (InstrE[14:12])
35 case (InstrE[14:12]) of:
36     3'b000 (beq): TakeBranchE = ZeroE;
37     3'b001 (bne): TakeBranchE = ~ZeroE;
38     3'b100 (blt): TakeBranchE = $signed(SrcAE) < $signed(SrcBE);
39     3'b101 (bge): TakeBranchE = $signed(SrcAE) >= $signed(SrcBE);
40     3'b110 (bltu): TakeBranchE = SrcAE < SrcBE;
41     3'b111 (bgeu): TakeBranchE = SrcAE >= SrcBE;
42     default: TakeBranchE = 0;
43
44 // 8. Decidir si se toma el salto
45 PCSrcE = (BranchE && TakeBranchE) || JumpE;

```

Listing 5: Pseudocódigo de la etapa Execute

4 Etapas Memory (MEM) y Writeback (WB)

Estas dos etapas finales son responsables de confirmar los resultados arquitectónicos, ya sea interactuando con la jerarquía de memoria o actualizando el banco de registros.

Teoría de Memoria y Sincronismo: La Memoria de Datos representa típicamente la caché L1 de datos en un procesador real. En nuestra implementación, la memoria es **síncrona para escritura y asíncrona para lectura**.

- **Escritura:** Solo ocurre en el flanco de subida del reloj cuando `MemWriteM` está en alto. Esto evita que señales inestables modifiquen el estado de la memoria.
- **Lectura:** Las memorias reales modernas suelen ser síncronas para lectura también (añadiendo latencia), pero nuestra implementación combinatorial simplificada asume que el dato se lee instantáneamente si la dirección (`ALUResultM`) es válida.

4.1 Componentes Activos

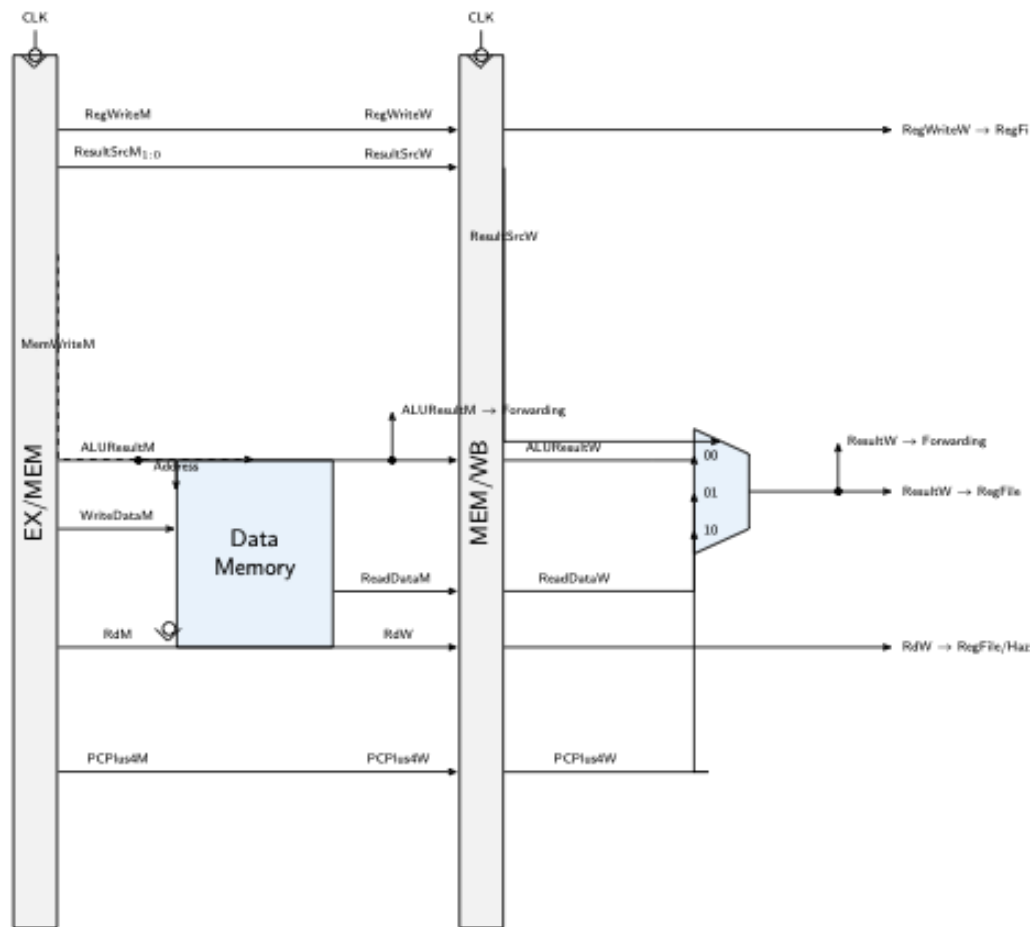


Figura 7: Etapas Memory y Writeback. La memoria de datos tiene escritura síncrona.

4.2 Pseudocódigo de ambas etapas

```

1 MEMORY(clk, reset, execute_signals):
2   // 1. Registro EX/MEM (flopr: siempre avanza)
3   always @(posedge clk):
4     if (reset) all_signals = 0;
5     else      all_signals = execute_signals;
6
7   // 2. Acceso a Memoria de Datos
8   ReadDataM = data_memory[ALUResultM / 4]; // Lectura combinacional
9   if (MemWriteM == 1):                     // Escritura síncrona
10    data_memory[ALUResultM / 4] = WriteDataM;
11
12 WRITEBACK(clk, reset, memory_signals):
13   // 1. Registro MEM/WB (flopr: siempre avanza)
14   always @(posedge clk):
15     if (reset) all_signals = 0;
16     else      all_signals = memory_signals;

```

```

17 // 2. Seleccionar resultado final
18
19 if (ResultSrcW == 2'b00):      ResultW = ALUResultW;    // Aritmetica
20 else if (ResultSrcW == 2'b01): ResultW = ReadDataW;     // Load (lw)
21 else if (ResultSrcW == 2'b10): ResultW = PCPlus4W;     // JAL

```

Listing 6: Pseudocódigo de Memory y Writeback

5 Programa ISA sin Dependencias (1.0 pts)

Este programa prueba que las instrucciones básicas del procesador (addi, add, sw, lw) funcionan correctamente cuando **no existen dependencias de datos entre instrucciones consecutivas**. Para lograrlo, se insertan instrucciones nop (no-operation) entre cada instrucción real, dando tiempo suficiente para que los datos se escriban en los registros antes de ser leídos por la siguiente instrucción.

5.1 Código Ensamblador

```

1 00: addi x2, x0, 5      // x2 = 5
2 04: nop                // Separador
3 08: nop
4 0C: nop
5 10: add  x3, x2, x2     // x3 = 10
6 14: nop
7 18: nop
8 1C: nop
9 20: sw   x3, 100(x0)    // MEM[100] = 10
10 24: nop
11 28: nop
12 2C: nop
13 30: lw   x4, 100(x0)   // x4 = MEM[100] = 10
14 34: nop
15 38: nop
16 3C: nop
17 40: add  x5, x4, x3     // x5 = 10 + 10 = 20
18 44: sw   x5, 200(x0)   // MEM[200] = 20

```

Listing 7: Programa 1: ISA sin dependencias (test_isa_sin_dependencias.mem)

5.2 Waveform del Programa 1



Figura 8: Waveform del Programa 1 (ISA sin dependencias). Las señales ForwardAE, ForwardBE, StallD y FlushE permanecen en 0 durante toda la ejecución.

5.3 Mostrar con Resultados Intermedios que el Programa Funciona Correctamente

Al observar el waveform, se verifica que:

- La señal `ALUResultE` muestra el valor `0x05` cuando `addi x2, x0, 5` pasa por `Execute`, y `0x0A` cuando `add x3, x2, x2` lo hace.
- La señal `ResultW` confirma que el valor `0x14` (20 en decimal) llega correctamente al final del pipeline como resultado de `add x5, x4, x3`.
- Las señales de la Hazard Unit (`ForwardAE`, `ForwardBE`, `StallD`, `FlushE`) permanecen en 0, demostrando que no se activó ningún mecanismo de protección. El programa fluye sin interrupciones.

6 Teoría de Hazard Unit y Evidencia de Fallos (0.5 pts)

6.1 Teoría de Control de Riesgos en Procesadores Segmentados

Al segmentar la ejecución de instrucciones, se solapan las etapas del datapath, rompiendo la ejecución estrictamente secuencial. Esto puede dar lugar a **riesgos** (hazards) que comprometen la integridad de los resultados si no se resuelven mediante hardware especializado:

- **Riesgos Estructurales (Structural Hazards):** Ocurren cuando dos instrucciones en etapas diferentes intentan acceder al mismo recurso físico al mismo tiempo. En este procesador se resuelven mediante una *Arquitectura Harvard*, que separa físicamente la memoria de instrucciones de la memoria de datos.
- **Riesgos de Datos (Data Hazards):** Surgen cuando una instrucción depende del resultado de una instrucción previa que aún no ha completado su escritura en el Register File. El caso más frecuente es la dependencia RAW (Read After Write).
- **Riesgos de Control (Control Hazards):** Producidos por instrucciones de salto condicional (`beq`) o absoluto (`jal`). Al leer la instrucción en `Fetch`, el procesador continúa cargando secuencialmente. Si el salto se resuelve como tomado en `Execute`, las instrucciones cargadas especulativamente en `Fetch` y `Decode` son incorrectas y deben descartarse.

6.2 Evidencia: Sin Hazard Unit los Programas Fallan

Para demostrar de forma empírica la necesidad crítica de la Hazard Unit, se procedió a desactivar el control de riesgos utilizando el módulo alternativo `hunit_disabled.v`. Este fuerza a cero los reenvíos y stalls. A continuación, se muestra el comportamiento de los programas de prueba bajo esta condición errónea:

6.2.1 Programa 2 SIN Hazard Unit (Forwarding)

Para este caso, se ejecuta el Programa 2, el cual intenta realizar operaciones aritméticas sobre registros cuyos resultados aún no han sido escritos en el banco de registros:

```

1 00: addi x2, x0, 5      // x2 = 5
2 04: addi x3, x0, 12     // x3 = 12
3 08: add  x4, x2, x3     // x4 = 5 + 12 = 17 (Riesgo RAW)
4 0C: sw   x4, 200(x0)    // MEM[200] = 17

```

Listing 8: Programa 2: test_forwarding.mem (para deshabilitado)

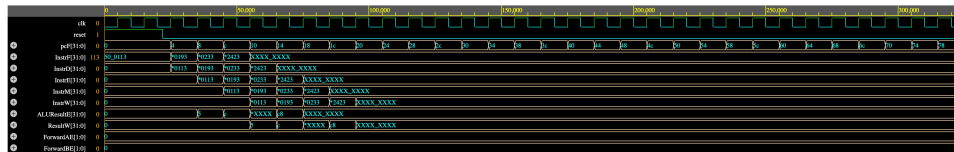


Figura 9: Programa 2 SIN Hazard Unit. La ALU opera con datos vacíos (0x00) porque los registros x2 y x3 aún no se han escrito cuando el add los necesita.

Interpretación: Sin forwarding, la instrucción `add x4, x2, x3` lee los registros x2 y x3 directamente del banco de registros. Como estas instrucciones previas (`addi x2` y `addi x3`) aún no han llegado a Writeback, la ALU recibe ceros y el resultado es incorrecto.

6.2.2 Programa 3 SIN Hazard Unit (Stalling)

En esta prueba se ejecuta el Programa 3, el cual presenta una dependencia de tipo Load-Use, requiriendo el uso inmediato de un dato cargado de memoria:

```

1 00: addi x2, x0, 5      // x2 = 5
2 04: sw   x2, 100(x0)    // MEM[100] = 5
3 08: lw   x3, 100(x0)    // x3 = MEM[100] = 5
4 0C: add  x4, x3, x2     // Load-Use Hazard (add usa x3 inmediatamente)
5 10: sw   x4, 200(x0)    // MEM[200] = 10

```

Listing 9: Programa 3: test_stalling.mem (para deshabilitado)

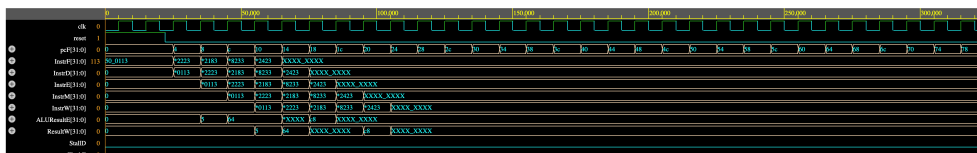


Figura 10: Programa 3 SIN Hazard Unit. El `add x4, x3, x2` usa un valor de x3 que aún no ha salido de la memoria.

Interpretación: El `lw x3, 100(x0)` carga un dato de memoria, pero el pipeline no se detiene. La instrucción `add x4, x3, x2` entra a Execute antes de que el dato esté disponible, produciendo un resultado corrupto.

6.2.3 Programa 4 SIN Hazard Unit (Flushing)

Para evaluar los riesgos de control se ejecuta el Programa 4, el cual realiza un salto condicional tomado que debería descartar las instrucciones siguientes cargadas de forma especulativa:

```

1 00: addi x2, x0, 5          // x2 = 5
2 04: beq x2, x2, 12         // Salta a linea 10 hex (siempre verdadero)
3 08: addi x3, x0, 1         // FLUSHED (no debe ejecutarse)
4 0C: addi x3, x0, 99        // FLUSHED (no debe ejecutarse)
5 10: add x4, x0, x2         // x4 = 5 (destino del salto)
6 14: sw x4, 200(x0)         // MEM[200] = 5

```

Listing 10: Programa 4: test_flushing.mem (para deshabilitado)

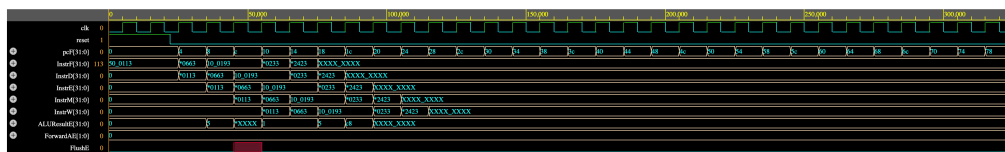


Figura 11: Programa 4 SIN Hazard Unit. Sin forwarding, las instrucciones previas al salto calculan incorrectamente, afectando la comparación del beq.

Interpretación: Sin la Hazard Unit, las instrucciones que preparan los operandos del beq no reciben los datos correctos vía forwarding. Esto puede causar que el salto se tome (o no) de forma errónea, ejecutando código que no debería ejecutarse.

7 Implementación del Hazard Unit (1.0 pts)

7.1 Descripción de Cambios en el Código

Para gestionar los riesgos de datos y de control de forma transparente para el programador (sin tener que inyectar NOPs manuales en el ensamblador), se implementó el módulo hunit (Hazard Unit) en el archivo Hazard Unit/hunit.v. Los principales cambios e implementaciones en el código incluyen:

1. **Diseño de la Unidad de Control de Riesgos:** Se diseñó un módulo puramente combinacional que monitorea las señales del datapath de forma continua y genera señales de bypass y de control en las etapas críticas.
2. **Estrategia de Pruebas de Fallo sin Dañar el Diseño:** Con el fin de demostrar la necesidad de la Hazard Unit sin modificar el procesador, se creó la unidad hunit_disabled.v. Este archivo tiene la misma firma (entradas y salidas) que la unidad real, pero inyecta un estado inactivo permanente (forzando reenvíos y stalls a 0), excepto por la redirección de saltos básicos para evitar que el PC entre en un ciclo infinito. La alternancia entre la simulación correcta y la errónea se realiza modificando una sola palabra en la instanciación del top-level (riscv_pipeline.v).

A continuación, se presentan los tres mecanismos principales de la unidad junto con sus diagramas y ecuaciones lógicas:

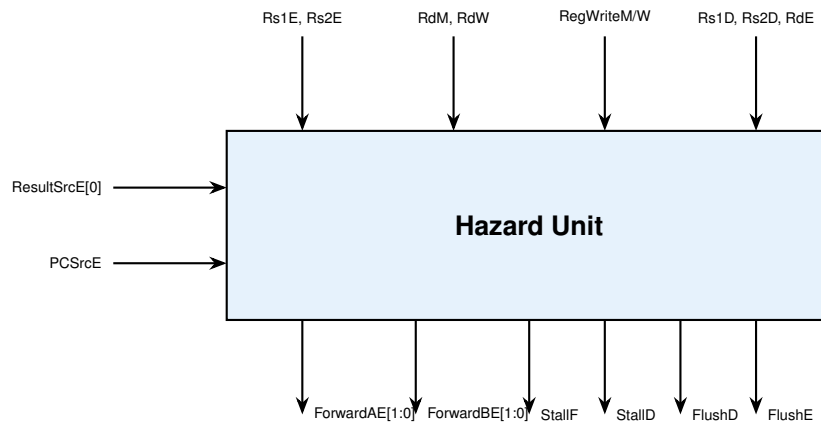


Figura 12: Diagrama de la Hazard Unit y sus múltiples conexiones.

7.2 Mecanismo 1: Forwarding (Reenvío de Datos)

El forwarding (o bypassing) es la solución de hardware más elegante para resolver los **Data Hazards** tipo RAW. Teóricamente, un resultado se calcula en la etapa Execute (ALU), pero arquitectónicamente no se escribe en el Register File hasta la etapa de Writeback (2 ciclos después). Si una instrucción posterior necesita este resultado inmediatamente, no puede leer el registro porque contiene un valor "viejo".

La Hazard Unit detecta que el registro destino de una instrucción en MEM o WB (*RdM* o *RdW*) coincide con los registros fuente de la instrucción actualmente en Execute (*Rs1E* o *Rs2E*). Al detectarlo, **cortocircuita** los multiplexores para inyectar el dato recién salido de la ALU (o de la Memoria) directamente en la entrada de la ALU de la instrucción actual, saltándose la espera por el Writeback.

```

1 // Forwarding para operando A (SrcAE)
2 if ((Rs1E == RdM) && RegWriteM && (Rs1E != 0)):
3     ForwardAE = 2'b10;    // Tomar resultado de Memory
4 else if ((Rs1E == RdW) && RegWriteW && (Rs1E != 0)):
5     ForwardAE = 2'b01;    // Tomar resultado de Writeback
6 else:
7     ForwardAE = 2'b00;    // Sin forwarding (usar registro)
8
9 // Forwarding para operando B (misma logica con Rs2E)

```

Listing 11: Lógica de Forwarding

7.3 Mecanismo 2: Stall (Load-Use Hazard)

El forwarding no puede resolver el Load-Use hazard (cuando un *lw* es seguido inmediatamente por una instrucción que lo usa). En ese caso, la Hazard Unit congela (stall) el PC y el IF/ID, e inserta una burbuja (flush) en el ID/EX.

```

1 // ResultSrcE[0] == 1 indica que la instruccion es un lw
2 lwStall = ResultSrcE[0] && ((Rs1D == RdE) || (Rs2D == RdE));
3
4 StallF = lwStall;    // Congelar PC (no avanza)
5 StallD = lwStall;    // Congelar IF/ID (mantener instruccion)
6 FlushE = lwStall;    // Vaciar ID/EX (insertar burbuja NOP)

```

Listing 12: Lógica de detección de Load-Use Hazard

7.4 Mecanismo 3: Flush (Control Hazard por Saltos)

Cuando se toma un salto ($PCSrcE=1$), las instrucciones que entraron especulativamente al Fetch y Decode deben descartarse, vaciando sus registros.

```

1 FlushD = PCSrcE;    // Vaciar IF/ID (descarta especulativa)
2 FlushE = lwStall || PCSrcE;    // Vaciar ID/EX

```

Listing 13: Lógica de flush por salto tomado

7.5 Especificación de Señales de Control de Riesgos

Problema	Solución	Señales	Penalización
Data Hazard (R-type)	Forwarding	ForwardAE, ForwardBE	0 ciclos
Load-Use Hazard	Stall + Forwarding	StallF, StallD, FlushE	1 ciclo
Control Hazard	Flush	FlushD, FlushE	2 ciclos

7.6 Diagrama de Datapath Final (con Hazard Unit)

El datapath completo con todas las compuertas de control de riesgos, los multiplexores de forwarding para SrcAE y SrcBE, y las señales de stall y clear de los registros del pipeline se muestra a continuación:

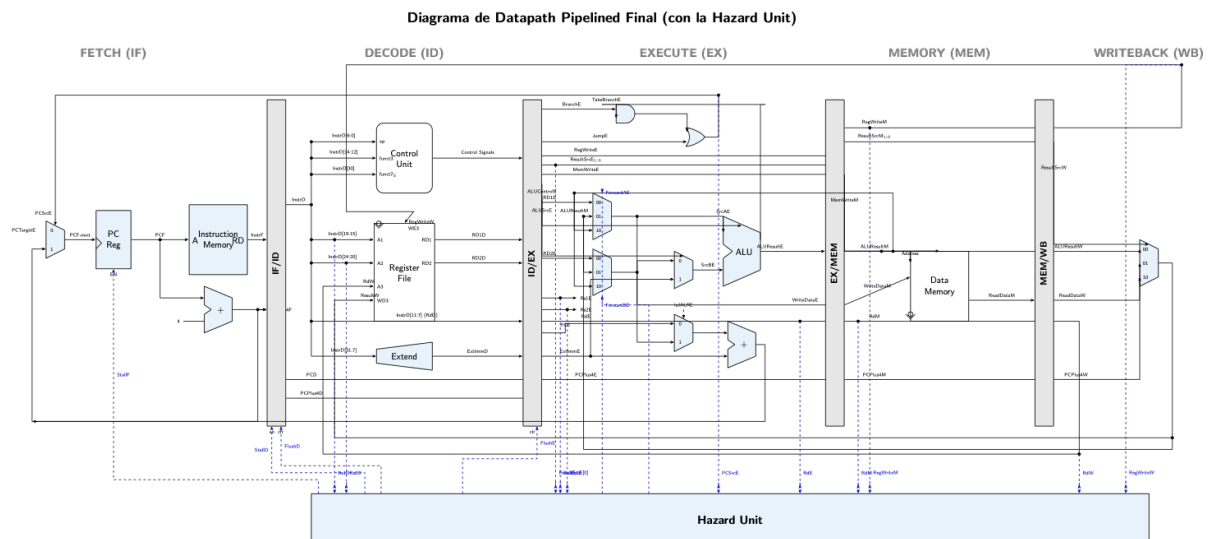


Figura 13: Diagrama del Datapath Pipelined completo del procesador (incluyendo la Hazard Unit).

El datapath final funciona correctamente ante cualquier dependencia de datos o de control debido a la incorporación de la Hazard Unit (unidad de control de riesgos) y los multiplexores de reenvío en la etapa Execute. La Hazard Unit supervisa continuamente y de forma combinacional los registros fuentes ($Rs1D$, $Rs2D$, $Rs1E$, $Rs2E$) y los compara con los registros de destino de instrucciones previas en las etapas de Execute (RdE), Memory (RdM) o Writeback (RdW) para tomar las siguientes acciones de control:

- **Forwarding (Reenvío):** Si hay un riesgo de datos del tipo RAW (Read After Write), la unidad activa las señales de selección $ForwardAE$ y/o $ForwardBE$. Esto hace que los multiplexores de 3 entradas ante la ALU tomen el dato más reciente desde Memory ($ALUResultM$) o Writeback ($ResultW$) en lugar de leer el valor obsoleto del banco de registros, logrando resolver el riesgo con 0 ciclos de penalización.
- **Stalling (Detección de Load-Use):** Cuando una instrucción de carga de memoria (lw) es seguida por otra que requiere su resultado en Execute, el reenvío no es suficiente ya que el dato se lee de memoria al final del ciclo de Memory. La unidad detecta esta condición y activa las señales de congelamiento $StallF$ y $StallD$ para detener la actualización del PC y del registro IF/ID por 1 ciclo, al mismo tiempo que activa $FlushE$ para inyectar una burbuja (instrucción NOP) en la etapa Execute, permitiendo al lw obtener el dato y realizar el reenvío en el ciclo siguiente.
- **Flushing (Saltos Condicionales y Saltos Directos):** Si se determina que un salto es tomado en Execute ($PCSrcE = 1$), las instrucciones cargadas especulativamente en las etapas Fetch y Decode deben ser descartadas para evitar que se ejecuten. La Hazard Unit activa las señales $FlushD$ y $FlushE$ para borrar los registros IF/ID e ID/EX (convirtiéndolos en NOPs), perdiendo 2 ciclos de pena-

lización pero garantizando la correcta ejecución del programa.

8 Programas de Prueba (CON Hazard Unit) (1.0 pts)

Al activar la Hazard Unit real (`hunit.v`), los tres programas producen resultados correctos.

8.1 Programa 2: Forwarding

El programa prueba que la Hazard Unit detecta dependencias RAW y reenvía datos desde Memory y Writeback hacia Execute.

```

1 00: addi x2, x0, 5      // x2 = 5
2 04: addi x3, x0, 12     // x3 = 12
3 08: add  x4, x2, x3      // x4 = 5 + 12 = 17 (Forwarding)
4 0C: sw   x4, 200(x0)    // MEM[200] = 17

```

Listing 14: Programa 2: test_forwarding.mem

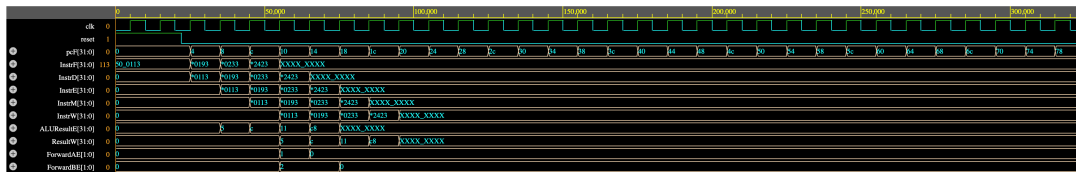


Figura 14: Programa 2 CON Hazard Unit. ForwardBE=10 reenvía x3 desde Memory; ForwardAE=01 reenvía x2 desde Writeback.

Interpretación: La Hazard Unit detecta que x3 (recién calculado, en etapa Memory) y x2 (en etapa Writeback) son necesarios por la instrucción `add x4, x2, x3`. Activa ForwardBE=10 y ForwardAE=01 para inyectar los valores correctos directamente a la ALU. El resultado es 0x11 (17 decimal), lo cual es correcto.

8.2 Programa 3: Stalling

El programa fuerza un Load-Use Hazard donde un `lw` es seguido inmediatamente por una instrucción que usa el dato cargado.

```

1 00: addi x2, x0, 5      // x2 = 5
2 04: sw   x2, 100(x0)    // MEM[100] = 5
3 08: lw   x3, 100(x0)    // x3 = MEM[100] = 5
4 0C: add  x4, x3, x2      // STALL: x3 aun no esta listo
5 10: sw   x4, 200(x0)    // MEM[200] = 10

```

Listing 15: Programa 3: test_stalling.mem

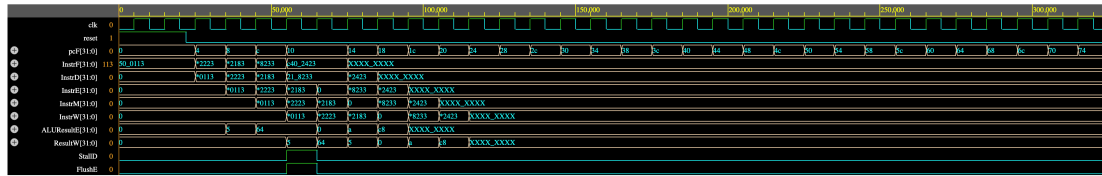


Figura 15: Programa 3 CON Hazard Unit. StallD congela Decode y Fetch durante 1 ciclo; FlushE inserta una burbuja en Execute.

Interpretación: La Hazard Unit detecta que el `lw x3` está en Execute (aún no ha accedido a memoria) y la instrucción `add x4, x3, x2` en Decode necesita `x3`. Activa `StallD=1` y `StallF=1` congelando el PC y el registro IF/ID por 1 ciclo. Simultáneamente, `FlushE=1` inserta una burbuja NOP en Execute. Al ciclo siguiente, el dato ya está disponible y se reenvía vía forwarding.

8.3 Programa 4: Flushing

El programa fuerza un Control Hazard con un salto condicional (`beq`) que es tomado.

```

1 00: addi x2, x0, 5      // x2 = 5
2 04: beq x2, x2, 12     // Salta a linea 10 hex (siempre verdadero)
3 08: addi x3, x0, 1     // FLUSHED (no debe ejecutarse)
4 0C: addi x3, x0, 99    // FLUSHED (no debe ejecutarse)
5 10: add x4, x0, x2     // x4 = 5 (destino del salto)
6 14: sw x4, 200(x0)    // MEM[200] = 5

```

Listing 16: Programa 4: test_flushing.mem

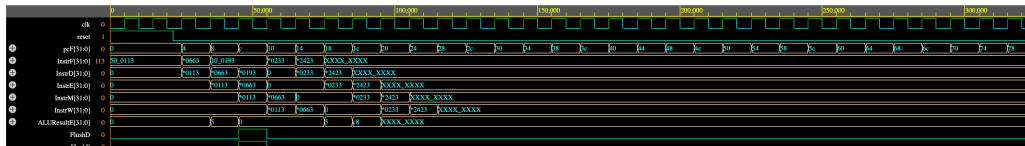


Figura 16: Programa 4 CON Hazard Unit. FlushE y FlushD se activan cuando `PCSrcE=1`, descartando las instrucciones especulativas.

Interpretación: El `beq x2, x2` se resuelve en la etapa Execute (ciclo 3). La ALU confirma que `x2 == x2` y activa `PCSrcE=1`. La Hazard Unit responde activando `FlushD` y `FlushE`, convirtiendo las instrucciones especulativas (`addi x3, x0, 1` y `addi x3, x0, 99`) en burbujas NOP (`0x00000000`). El PC salta directamente a `0x10` y la ejecución continúa correctamente.

8.4 Programa Extra: Set Completo de Operaciones y Validación de la ISA y Hazard Unit

Para validar de forma integral el soporte de todas las operaciones de la ALU implementadas (incluyendo la red de inmediatos y desplazamientos) y todos los mecanismos de

control (bifurcaciones condicionales y saltos incondicionales con flushing), se diseñó un programa adicional exhaustivo. Este programa ejecuta de manera consecutiva el repertorio RV32I completo, permitiendo visualizar riesgos de datos (RAW Hazards resueltos por reenvío) y riesgos de control (Control Hazards resueltos por vaciado de instrucciones).

```

1 00: addi x2, x0, 15 // x2 = 15 (0x0F)
2 04: addi x3, x0, 5 // x3 = 5
3 08: add x4, x2, x3 // x4 = 15 + 5 = 20 (0x14) (RAW Hazard en x2 y x3 ->
    Forwarding)
4 0C: sub x5, x2, x3 // x5 = 15 - 5 = 10 (0x0A) (RAW Hazard en x2 y x3 ->
    Forwarding)
5 10: and x6, x2, x3 // x6 = 15 & 5 = 5 (0x05)
6 14: or x7, x2, x3 // x7 = 15 | 5 = 15 (0x0F)
7 18: xor x8, x2, x3 // x8 = 15 ^ 5 = 10 (0x0A)
8 1C: sll x9, x2, x3 // x9 = 15 << 5 = 480 (0x1E0)
9 20: srl x10, x2, x3 // x10 = 15 >> 5 = 0
10 24: sra x11, x2, x3 // x11 = 15 >>> 5 = 0
11 28: slt x12, x2, x3 // x12 = (15 < 5 signed) ? 1 : 0 = 0
12 2C: sltu x13, x2, x3 // x13 = (15 < 5 unsigned) ? 1 : 0 = 0
13 30: sw x4, 100(x0) // Mem[100] = x4 = 20 (0x14) (RAW Hazard en x4 ->
    Forwarding)
14 34: lw x14, 100(x0) // x14 = Mem[100] = 20 (0x14)
15 38: lui x15, 0x12345 // x15 = 0x12345000 (Carga de inmediato alto, U-type)
16 3C: slli x16, x14, 2 // x16 = 20 << 2 = 80 (0x50) (RAW Hazard en x14 ->
    Forwarding)
17 40: xori x17, x16, 10 // x17 = 80 ^ 10 = 90 (0x5A) (RAW Hazard en x16 ->
    Forwarding)
18 44: srli x18, x16, 2 // x18 = 80 >> 2 = 20 (0x14) (RAW Hazard en x16)
19 48: srai x19, x16, 2 // x19 = 80 >>> 2 = 20 (0x14) (RAW Hazard en x16)
20 4C: ori x20, x18, 5 // x20 = 20 | 5 = 21 (0x15) (RAW Hazard en x18)
21 50: andi x21, x18, 8 // x21 = 20 & 8 = 0 (0x00) (RAW Hazard en x18)
22 54: bne x18, x19, 0x8C // no tomado (20 == 20). No altera el flujo del PC.
23 58: beq x18, x19, 0x64 // tomado (20 == 20). Salta a 0x64. (Flushea 0x5C y
    0x60)
24 5C: addi x5, x0, 99 // FLUSHED (burbuja NOP)
25 60: addi x5, x0, 99 // FLUSHED (burbuja NOP)
26 64: addi x22, x0, -10 // x22 = -10 (0xffffffff6)
27 68: addi x23, x0, 5 // x23 = 5
28 6C: blt x22, x23, 0x78 // tomado (-10 < 5, signed). Salta a 0x78. (Flushea
    0x70 y 0x74)
29 70: addi x5, x0, 99 // FLUSHED
30 74: addi x5, x0, 99 // FLUSHED
31 78: bge x23, x22, 0x84 // tomado (5 >= -10, signed). Salta a 0x84. (
    Flushea 0x7C y 0x80)
32 7C: addi x5, x0, 99 // FLUSHED
33 80: addi x5, x0, 99 // FLUSHED
34 84: jal x1, 0x94 // Salto incondicional a 0x94. x1 = PC+4 = 0x88. (
    Flushea 0x88)
35 88: addi x5, x0, 99 // FLUSHED
36 8C: nop // Trampa de error (no ejecutada)
37 90: nop // Trampa de error (no ejecutada)
38 94: addi x24, x0, 172 // x24 = 172 (0xAC)
39 98: jalr x25, x24, 0 // Salto indirecto a x24 + 0 = 0xAC. x25 = PC+4 = 0x9C
    . (Flushea 0x9C, 0xA0, 0xA4, 0xA8)
40 9C: addi x5, x0, 99 // FLUSHED

```

41 AC: addi x26, x0, 77 // x26 = 77 (0x4D) (Destino del salto indirecto JALR)

Listing 17: Programa Extra: test_remaining_ops.mem

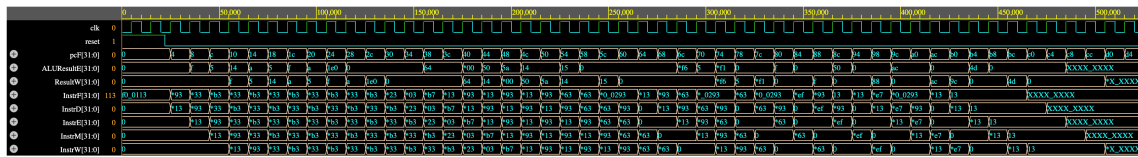


Figura 17: Formas de onda del Programa Extra. Se verifica el correcto comportamiento temporal y la resolución de dependencias de datos (forwarding) y de control (flushing) para todas las instrucciones RV32I.

Interpretación:

- **Verificación de Operaciones ALU e Inmediatos:** Se valida la correcta ejecución de operaciones aritméticas con registro e inmediato. Por ejemplo:
 - En el ciclo 16, la instrucción `slli` calcula `0x50` (80 decimal) a partir de `x20` desplazado 2 bits.
 - En el ciclo 17, la instrucción `xori` calcula `0x5A` (90 decimal) a partir de `80 ^ 10`.
 - La instrucción `lui` coloca el inmediato alto `0x12345000` en el registro `x15` (mostrado en `ResultW` en el ciclo 18).
- **Control de Bifurcaciones (Branches):**
 - La instrucción `bne` en `0x54` no toma el salto al ser operandos iguales, permitiendo que el PC avance secuencialmente a `0x58`.
 - La instrucción `beq` en `0x58` sí toma el salto, lo que provoca que `PCSrcE` se active, y la Hazard Unit aplique un `FlushD` y `FlushE` para limpiar las instrucciones especulativas en `0x5C` y `0x60`, saltando el PC directamente a `0x64`.
 - Las instrucciones de branch comparativos con signo `blt` (menor que) y `bge` (mayor o igual) se evalúan correctamente, redirigiendo el PC a sus destinos con la consecuente inserción de burbujas en el pipeline.
- **Salto Incondicionales ('jal' y 'jalr'):**
 - `jal` en `0x84` calcula su destino relativo en la etapa `Execute` y salta a `0x94`, guardando el enlace `PC+4 = 0x88` en `x1` en el ciclo 37.
 - `jalr` en `0x98` calcula su destino basándose en el registro `x24` (que contiene `0xAC`), saltando directamente a la instrucción en `0xAC` (que carga `77` en `x26`) y guardando el enlace de retorno `PC+4 = 0x9C` en `x25` en el ciclo 41.

9 Extension RVC (16-bits)

La extension estandar RVC (Compressed Instructions) de la arquitectura RISC-V tiene como objetivo reducir el tamaño del código permitiendo que las instrucciones más frecuentes sean codificadas en 16 bits (2 bytes) en lugar de los 32 bits tradicionales.

9.1 El Reto Arquitectónico y la Solución

En nuestro diseño original pipelined de 5 etapas, la CPU estaba diseñada asumiendo que el universo entero estaba hecho de bloques de 32 bits. Para no tener que reescribir todo el procesador (lo cual habría destruido la Hazard Unit), se adoptó una solución sumamente elegante: **ocultarle al pipeline que existen instrucciones de 16 bits**.

La meta fue crear un descompresor combinacional dentro de la primera etapa (Fetch). Cuando llega una instrucción de 16 bits, este traductor la convierte instantáneamente en su equivalente de 32 bits antes de que pase a la etapa Decode. Así, Decode siempre ve instrucciones estandar de 32 bits.

9.2 Modificaciones al Código

- **Alineación en Memoria:** Ahora la memoria se lee en bloques de 32 bits estandar, pero introducimos un multiplexor de hardware que revisa si el Program Counter está desalineado (bit `PCF[1]`).
- **Descompresor (`decompressor.v`):** Este módulo extrae los 16 bits más bajos, revisa si es una instrucción comprimida, e infla los bits a su versión RV32I.
- **El Engano al Pipeline:** Se utiliza un multiplexor para elegir entre la instrucción leída original o la versión expandida por el descompresor.
- **Avance Dinámico del PC:** El Program Counter ya no avanza 4 bytes rigidamente, sino que avanza +2 si la instrucción fue de 16 bits o +4 si fue de 32 bits.

9.3 Justificación: Unidad de Riesgos (Hazard Unit)

No se modificó ni una sola línea de código en la `hunit.v`. Dado que el descompresor se ubicó antes del registro `IF/ID`, la etapa Decode siempre recibe un bloque de 32 bits perfecto. La Hazard Unit procesa registros y emite señales de forwarding o stall exactamente de la misma manera que lo hacía antes.

9.4 Pruebas de la Extension RVC

9.4.1 Test RVC Basico (`test_rvc_basic.mem`)



Figura 18: Formas de onda del test basico RVC. Se observa la mezcla de instrucciones `c.addi`, `add` normal y `c.sub`.

Interpretacion de la forma de onda:

- **El salto de 2 en 2:** En la senal `PCF`, se puede observar claramente la secuencia de avance `0 ->2 ->4 ->8`. El salto inicial de 2 en 2 bytes comprueba visualmente que el PC esta reconociendo la instruccion de 16 bits (`c.addi`) y avanzando adecuadamente.
- **Senal de Compresion:** La senal interna `is_compressed` se eleva a alto (1) durante los ciclos donde el PC apunta a direcciones alineadas a media palabra.

9.4.2 Test RVC de Memoria (`test_rvc_memory.mem`)

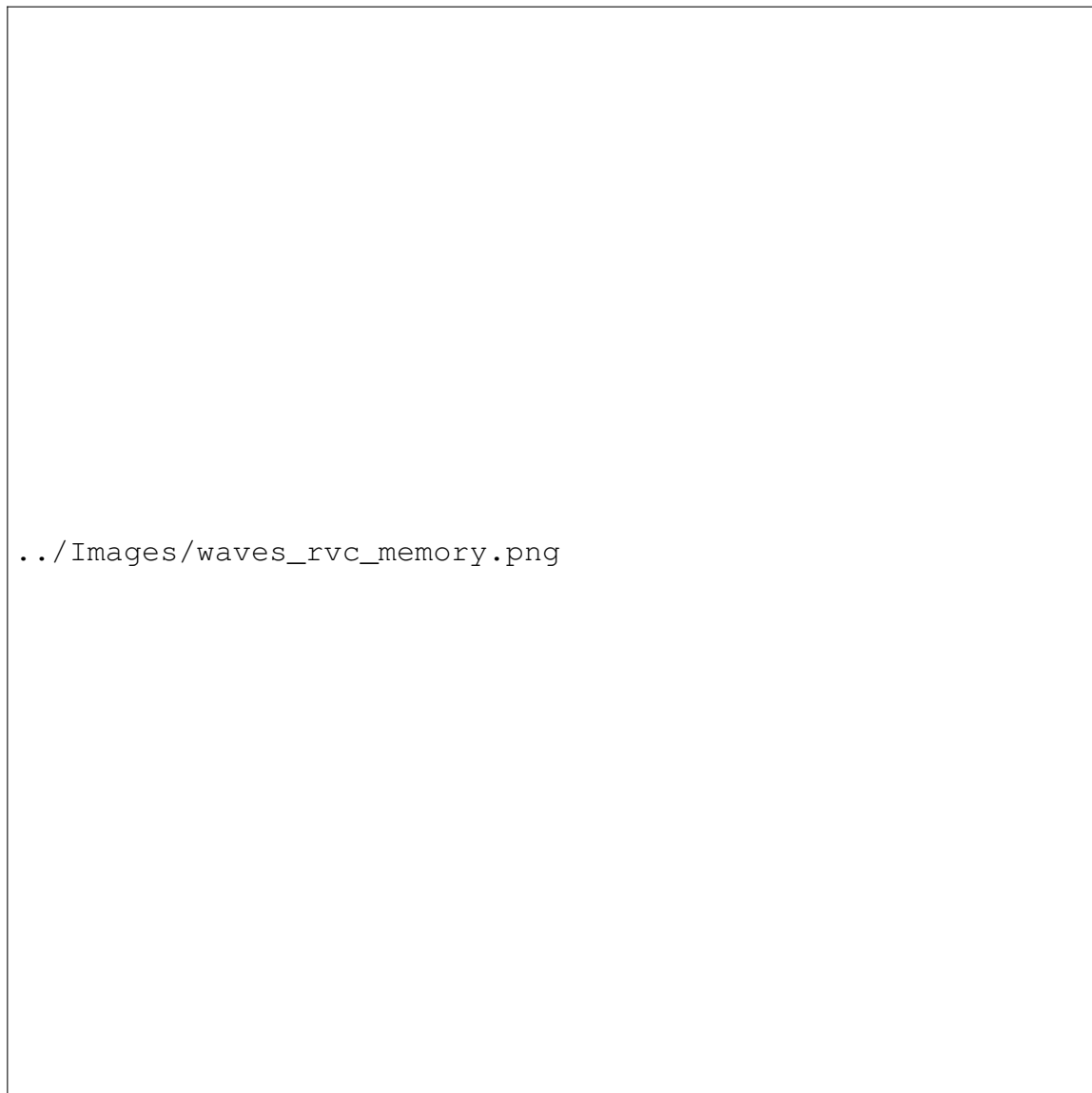


Figura 19: Formas de onda del test de memoria RVC. Se evalúa el funcionamiento de las instrucciones `c.lw` y `c.sw`.

Interpretación de la forma de onda:

- A pesar de que las instrucciones en el código fuente ocupan solo 16 bits, la etapa Memory interactúa con Data Memory transfiriendo 32 bits completos, tal como lo demuestra la validación de las escrituras y lecturas exitosas en memoria. Las señales `MemWrite` se activan en los ciclos correctos.

9.4.3 Test RVC de Saltos (`test_rvc_branches.mem`)

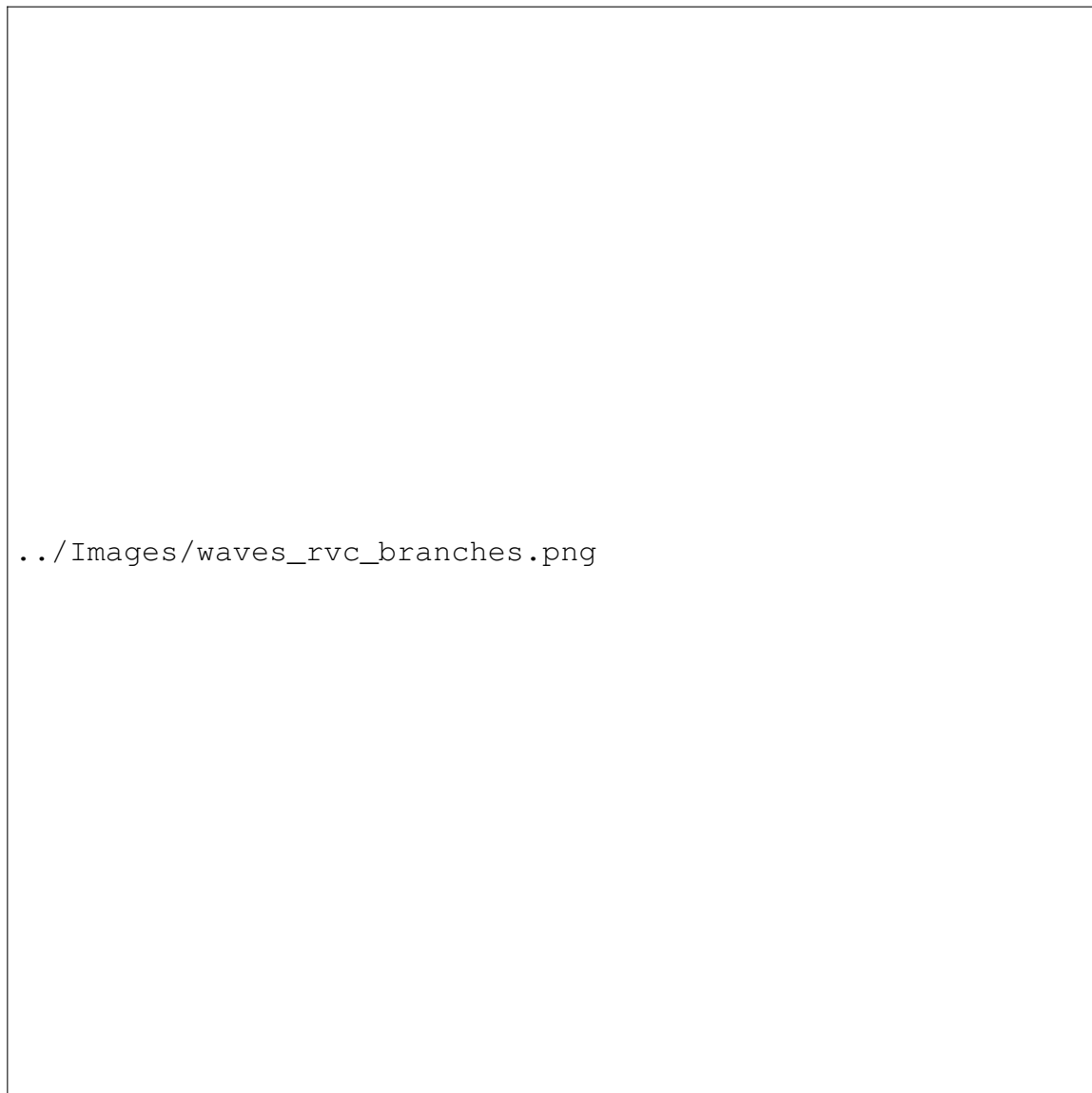


Figura 20: Formas de onda del test de saltos RVC. Ejecucion de branches condicionales `c.beqz` y `c.bnez`.

Interpretacion de la forma de onda:

- Se visualiza el correcto calculo de offsets (saltos relativos al PC) para instrucciones comprimidas. A pesar de los 16 bits, el modulo Execute genera el Branch Target adecuado, y la Hazard Unit logra hacer el `Flush` de manera transparente ante un salto tomado.

10 Jerarquia de Archivos


```

|-- riscv_pipeline.v           (Top module)
|-- tb.v                      (Testbench)
|-- run_sim.sh                (Script de simulacion)
|-- Datapath/
|   |-- Fetch/               fetch.v, imem.v, decompressor.v
|   |-- Decode/              decode.v, regfile.v
|   |-- Execute/             execute.v
|   |-- Memory/              memory.v, dmem.v
|   |-- WriteBack/           writeback.v
|-- Controller/               controller.v
|-- Hazard Unit/              hunit.v, hunit_disabled.v
|-- Utils/                    flopr.v, flopenr.v, flopenrc.v,
|                               mux2.v, mux3.v, adder.v, alu.v, extend.v
|-- Tests/
|   |-- test_isa_sin_dependencias.mem  (Programa 1)
|   |-- Entrega2_RVC/
|       |-- test_rvc_basic.mem          (Test RVC 1)
|       |-- test_rvc_branches.mem       (Test RVC 2)
|       |-- test_rvc_memory.mem         (Test RVC 3)
|-- Waves/
|   |-- waves_isa.vcd, waves_forwarding.vcd,
|   |   waves_rvc_basic.vcd, waves_rvc_branches.vcd,
|   |   waves_rvc_memory.vcd
|-- Informe_LaTeX/ informe_pipeline.tex

```

11 Conclusiones

1. **Pipeline vs. Single-Cycle:** La arquitectura pipelined incrementa significativamente el rendimiento dividiendo la ejecución en 5 etapas solapadas, al coste de añadir complejidad con la Hazard Unit.
2. **Necesidad de la Hazard Unit:** Como se demostró en la Sección 4, sin la Hazard Unit los programas con dependencias producen resultados incorrectos. Los mecanismos de forwarding, stalling y flushing son indispensables para garantizar la correctitud del procesador.
3. **Forwarding:** Resuelve la mayoría de los Data Hazards sin penalización (0 ciclos perdidos), cortocircuitando resultados desde Memory y Writeback hacia Execute.
4. **Stalling:** El único caso que requiere detener el pipeline es el Load-Use Hazard, con una penalización mínima de 1 ciclo.
5. **Flushing:** Los saltos tomados provocan una penalización de 2 ciclos al descartar instrucciones especulativas, pero garantizan la integridad del flujo de control.
6. **Modularidad:** El diseño modular (cada etapa en su propio archivo) facilita la comprensión, depuración y extensión futura del procesador.